

Rapport de Projet

Compression de fichiers



Matias Lievain–Paredes, Eduardo Cobos Fernandez

Table des matières

1. Introduction	3
2. Répartition du travail	4
3. Construction de l'arbre de Huffman	4
4. Compression et décompression du texte	5
5. Jeux de tests effectués	6
6. Extraits de code	7
7. Conclusion	9

Introduction

De nos jours, le volume de données produit à chaque instant est énorme et ne cesse de croître de plus en plus. Afin de pouvoir gérer ce flux énorme de données, la compression de fichiers s'est imposée comme une technologie essentielle. Sans elle nos disques durs déborderaient d'informations au bout de très peu de temps d'usage, les transferts de données prendraient des heures et on ne pourra plus avoir des systèmes de communication aussi performants et le coût énergétique et environnemental de l'informatique augmenterait considérablement à cause du stockage et le transfert de fichiers volumineux.

Vu cette importance, depuis les années 50 un très grand nombre d'équipes de développeurs a travaillé sur divers algorithmes de compression en aboutissant à des algorithmes tels celui de Shannon-Fano (1949), celui nommé LZ77 (1977) ou l'algorithme de Huffman (1951).

Dans ce projet, on s'est intéressé à ce dernier qui exploite la fréquence des caractères afin de représenter les données d'un fichier avec un minimum de bits, en attribuant des codes courts aux éléments fréquents et des codes plus longs aux éléments moins fréquents. Et on a implémenté cet algorithme en utilisant le langage de programmation Ocaml.

Dans ce rapport nous détaillerons les étapes de l'implémentation de cet algorithme en commençant par expliquer la création de l'arbre de huffman, puis comment on s'en ai servi dans nos fonctions qui compressent et décompressent un fichier donné, finalement on présentera aussi le jeu de tests avec lequel on s'est assuré que notre implémentation est correcte.

Répartition du travail

Concernant la répartition du travail dans le projet, nous avons choisi de travailler ensemble sur un même ordinateur. On a considéré que cette approche était la plus efficace car elle nous permettait de réfléchir à deux au codage des fonctions et aux choix des structures de données qu'il fallait implémenter. De plus, la plupart des fonctions dépendaient les unes des autres ce qui compliquait la division du travail car un membre du binôme n'aurait pas su comment l'autre avait codé les fonctions qu'il devait utiliser.

On a donc travaillé à deux sur le même poste de travail en effectuant les commits sur Framagit directement depuis cet ordinateur. Une fois l'intégrité du code finie, nous avons réparti le travail pour la rédaction du rapport : où chacun s'est chargé d'expliquer différents aspects du projet, permettant ainsi d'avancer plus rapidement.

Construction de l'arbre de Huffman

Le point principal de l'algorithme de Huffman repose sur l'utilisation de ce qu'on appelle un arbre de Huffman. C'est un arbre binaire valué aux feuilles (c'est-à-dire que chaque nœud de l'arbre possède soit deux fils soit aucun et où seules les feuilles) où les feuilles représentent les caractères du texte à compresser.

Le but sera de représenter chaque caractère du texte que l'on veut compresser par un code binaire qui sera déterminé par le chemin parcouru depuis la racine de l'arbre jusqu'à la feuille où se situe le caractère (avec la convention que prendre le sous-arbre gauche correspond à un 0 et prendre le sous-arbre droit correspond à un 1). La position de chaque caractère dans l'arbre ne sera donc pas choisie au hasard, elle sera choisie en fonction de la fréquence d'apparition dans le texte du caractère (comme ça les codes les plus utilisés seront plus courts que ceux qui s'utilisent moins).

Dans notre implémentation, la construction de l'arbre commence par un parcours complet du fichier d'entrée pour calculer la fréquence d'apparition de chaque caractère. Ces fréquences sont stockées dans un tableau de taille 257, où chaque case correspond à un octet spécifique, avec une case supplémentaire pour gérer d'éventuels caractères spéciaux. Une fois ce tableau rempli, nous extrayons uniquement les caractères qui apparaissent au moins une fois dans le fichier et nous les regroupons dans une liste sous forme de paires composées du caractère et de sa fréquence.

À partir de cette liste, nous initialisons les feuilles de l'arbre en créant un nœud pour chaque paire. Ensuite, nous construisons l'arbre de manière récursive en suivant l'algorithme de Huffman : à chaque itération, nous identifions les deux nœuds ayant les fréquences les plus faibles, que nous combinons sous un nouveau nœud interne dont le poids est égal à la somme des deux fréquences. Ce nœud remplace les deux nœuds d'origine dans la liste, et l'opération est répétée jusqu'à ce qu'il ne reste plus qu'un seul nœud, qui devient la racine de l'arbre.

Compression et décompression du texte

Si l'extension du fichier donné dans la ligne de commande est '.hf', alors on décomprime le fichier donné en paramètre, sinon on le comprime.

Pour comprimer le texte, nous créons d'abord l'arbre de Huffman associé à celui-ci grâce à la file de priorité et à l'algorithme de Huffman. Ensuite, nous convertissons cet arbre en une liste de paires (caractère, code de ce caractère dans l'arbre) et nous sérialisons l'arbre dans le fichier comme il nous a été indiqué de le faire. Nous parcourons ensuite tout le texte par récurrence en écrivant chaque caractère du texte à comprimer dans le fichier

comprimé, en le remplaçant par sa valeur que l'on trouve maintenant dans la liste de paires.

Si le fichier s'appelait avant compression fichier.extension, le nom du fichier comprimé sera fichier_extension.hf. Nous pouvons ainsi compresser tous types de fichiers et non seulement des fichiers de type '.txt'.

On va pouvoir maintenant récupérer l'extension du fichier d'origine lors de la décompression en cherchant le string se trouvant entre le dernier '_' et le '.' suivant. Le fichier décompressé se nommera fichier_decomprese.extension.

Pour la décompression du texte, nous réalisons d'abord une désérialisation de l'arbre. Pour cela, on crée d'abord la structure de l'arbre avec des -1 comme valeur aux feuilles, puis on récupère les vraies valeurs de ces feuilles récursivement. Ensuite, on lit bit par bit le flux jusqu'à arriver à une feuille qui est donc le caractère décompressé par l'algorithme de Huffman et on écrit ce caractère dans le fichier décompressé. On répète cette opération jusqu'à ce qu'il n'y ait plus de caractères à lire dans le fichier compressé.

Jeux de tests effectués

Afin de vérifier que notre implémentation de l'algorithme était correcte, nous avons réalisé une grande variété de tests qui peuvent être séparés en deux groupes. D'une part, les petits tests, effectués au fur et à mesure de la programmation qui servaient à tester le bon fonctionnement du code que l'on écrivait avant de passer à coder la fonction suivante qui utilisait ce code. D'autre part, les grands tests qui ont été lancés une fois l'implémentation complète, consistant à télécharger ou créer des fichiers, les compresser puis décompresser et regarder que l'on retrouvait le fichier d'origine.

Les petits tests étaient effectués grâce à des affichages insérés dans le main() et dans des places stratégiques au sein des fonctions que l'on

voulait tester. Ils nous ont permis notamment de vérifier que la création de la file de priorité se faisait correctement et que les arbres de Huffman que les fonctions créaient étaient corrects (on confirmait ceci en comparant ceux donnés par le code à ceux que l'on faisait à main). Il est important de préciser que les lignes de code qui se chargeaient des affichages étaient supprimées après les tests car elles ajoutaient des lignes de code en trop et on avait plus besoin des affichages.

Pour les tests plus importants qui vérifiaient la bonne compression et décompression des fichiers, nous avons commencé par des petits fichiers écrits à la main. On les comprimait et on vérifiait si ce qui était à l'intérieur du nouveau fichier semblait correct, puis on les décomprimait et on observait si la version qu'on trouvait correspondait à l'originale. Ensuite, on a reproduit ces tests, mais avec des fichiers plus grands de taille téléchargés d'internet et avec des fichiers qui n'étaient pas forcément des .txt. Finalement, on a codé une fonction qui lance des tests dans huffman.ml et qui est appelé lorsqu'on écrit “ ./huff.exe --tests ” à la ligne de commande. Cette fonction comprime puis décomprime certains fichiers spécifiques et vérifie que la version trouvée est la même que la version originelle avec une fonction que nous avons codée.

Extraits de code

```
(**[creation_arbre tab] crée l'arbre cherché en prenant en argument le tab réduit
@param le tableau de paires (nombre d'occurrence, feuille du caractère)
@return l'arbre de Huffman associé au texte*)
(file_perm * tree) list -> tree
let rec creation_arbre tab =
  match tab with (*Le plus petit va toujours vers la gauche*)
  | [] -> Leaf (-1)(*Arbre vide*)
  | [(ocu, abr)] -> abr (*retourne l'arbre cherché*)
  | (ocu, abr1)::(ocu2, abr2)::t -> let min1 = min_tab tab in
                                   let tt = elimine_min (tab) in
                                   let min2 = min_tab tt in
                                   creation_arbre((fst(min1)+fst(min2), (add (snd min1) (snd min2))))::(elimine_min tt))
```

Cette fonction crée l'arbre de Huffman à partir d'une liste de paires (occurrence, feuille de la valeur du caractère). Pour cela, on effectue un parcours de liste de manière récurrente. Si la liste ne possède aucun élément, cela veut dire que l'archive de départ est vide. Si elle possède un seul élément, c'est-à-dire qu'on a déjà créé l'arbre voulu. Sinon, on

prend l'élément avec le moins d'occurrences dans la liste, puis l'élément avec le moins d'occurrences dans la liste privée de l'élément que l'on vient de prendre et on crée maintenant une paire qui somme le nombre d'occurrences de ces deux éléments d'une part et qui crée un arbre entre ces deux éléments d'une autre. On remet ce nouvel élément dans la liste privée de ces deux éléments, puis on parcourt de nouveau la liste.

```

(**[deserialise_abr fichier istream] fonction qui rend l'arbre à partir du fichier compressé
@param istream le flux d'entrée
@return la désérialisation de l'arbre de Huffman associé au texte compressé
@raise Invalid_stream si le flux n'est pas valide
*)
istream -> tree
let deserialise_abr istream =
  (*fonction qui rend l'arbre à partir du fichier compressé*)
  let rec struct_abr istream =
    (*fonction qui rend la structure de l'arbre avec des Leaf(-1) aux feuilles*)
    let bit = read_bit istream in
    match bit with
    | 0 -> Leaf(-1)
    | 1 -> let sabrg = struct_abr istream in
            let sabrd = struct_abr istream in
            Node(sabrg, sabrd)
    | _ -> raise Invalid_stream
  in

  let rec lire_feuilles abr istream =
    (*fonction qui rend un nouvel arbre avec les valeurs des caractères comprimés*)
    match abr with
    | Leaf _ -> let feuille = read_byte istream in
                Leaf(feuille)
    | Node(sabrg, sabrd) -> let gauche = lire_feuilles sabrg istream in
                            let droite = lire_feuilles sabrd istream in
                            Node(gauche, droite)
  in

  let res = struct_abr istream in
  let abr = lire_feuilles res istream in
  abr

```

Cette fonction désérialise l'arbre de Huffman à partir du flux du fichier compressé. Pour cela, on crée d'abord la structure de l'arbre en mettant -1 comme valeur aux feuilles. Cela se fait de manière récurrente par lecture de bits. Si le bit trouvé est un 0, alors on est sur une feuille et on renvoie un Leaf(-1), sinon, on parcourt le reste du flux pour faire l'arbre gauche puis pour faire l'arbre droit. Cette récursion est terminale puisque l'arbre est sérialisé de manière correcte. Il nous reste ensuite à remplacer les valeurs -1 par la vraie valeur des caractères. Pour cela, on parcourt notre structure d'arbre de manière récurrente, si on est sur une feuille, on rajoute la valeur du byte lu, sinon on remplit d'abord le sous arbre gauche puis le sous arbre droit.

Conclusion

En conclusion, ce projet nous a permis de consolider et d'approfondir nos compétences en programmation OCaml, tout en nous permettant de mieux comprendre les algorithmes de compression de fichiers, notamment l'algorithme de Huffman. En partant d'une approche initiale différente, nous avons dû repartir de zéro et réorganiser notre code, ce qui nous a permis d'appréhender l'importance de la récursion dans la simplification du développement et de l'optimisation du code.

Nous avons finalement noté que l'algorithme n'offre pas les mêmes résultats de compression pour tous les types de fichiers. Par exemple, pour certains fichiers comme les .pdf, le taux de compression est moins élevé comparé à des fichiers de type .txt.

Il serait donc intéressant de perfectionner cet algorithme pour qu'il soit plus efficace avec les compressions d'autres types de fichiers tels les pdf, des images ou les archives vidéos.